

Module 2

Process Management

2.1 INTRODUCTION

Earlier, there was a limitation of loading only one program into the main memory for execution at a time. This program was very multifaceted and resourceful as it had access to all the resources of the computer, such as memory, CPU time, I/O devices, and so on. As time went by, newer techniques incorporated a variety of novel and powerful features that dramatically improved the efficiency and functionality of the overall system.

Modern computer systems corroborate multiprogramming, which allows a number of programs to reside in the main memory at the same time. These programs can be executed concurrently thereby requiring the system resources to be shared among them. Multiprogrammed systems need to distinguish among the multiple executing programs, and this is accomplished with the concept of a process (also called **task** on some systems).

When multiple processes run on a system concurrently and more than one processes require the CPU at the same time, then it becomes essential to select any one process to which the CPU can be allocated. To serve this purpose, **scheduling** is required. Moreover, the multiple processes running on a system also need to intercommunicate in order to reciprocate some data or information. This kind of intercommunication between several processes is referred to as **inter-process communication (IPC)**.

2.2 PROCESS CONCEPT

2.2.1 The Process

A process is a program under execution or an executing set of machine instructions. It can be either a **system process** executing the system's

code, or a **user process** executing the user's code.

There is only a hairline difference between the program and the process in the sense that a program is a passive entity that does not initiate anything by itself whereas a process is an active entity that performs all the actions specified in a particular program.

A process comprises not only the program code (known as **text section**) but also a set of global variables (known as **data section**) and the **process control block (PCB)**.

There can be either one-to-one or one-to-many relationship between programs and processes.

A **one-to-one** relationship exists in case only a single instance of a program is running on the system.

On the other hand, if the multiple instances of a single program are running simultaneously or when a concurrent program (a program that requires some of its parts to be executed concurrently) is being run, there exists **one-to-many relationship** between programs and processes. In this case, the text section of the multiple instances will be same but the data section will be different.

One important thing to notice about the processes is that some processes involve higher computation than I/O operations thereby demanding greater use of the CPU than I/O devices during their lifetime.

Such processes where the speed of execution is governed by the CPU are called **CPU-bound** or **compute-bound**. Contrastive to this are some processes that involve a lot of I/O operations during their lifetime. Such processes where the speed of execution is governed by the I/O device, not by the CPU, are called **I/O-bound**.

2.2.2 Process Attributes

In operating systems, a process is an instance of a computer program that is being executed. Each process has a set of attributes that define its state, resources, and behavior. These attributes are crucial for the OS to manage and control the execution of processes.

Key attributes include process ID, program counter, process state, priority, registers, open files, and I/O devices.

Core Process Attributes:

- **Process ID (PID):** A **unique identifier** assigned to each process by the operating system, allowing for its distinct identification.
- **Program Counter (PC):** **Stores the address of the next instruction to be executed by the process.** It's essential for resuming execution after a context switch.
- **Process State:** Indicates **the current status of a process**, such as new, ready, running, waiting, or terminated.
- **Priority:** Determines **the importance of a process**, influencing its **access to the CPU**. Higher priority processes receive preferential treatment.
- **General-Purpose Registers:** These registers **hold data that the process is actively using during execution.**
- **List of Open Files:** Keeps track of the **files currently being accessed** by the process.
- **List of Open I/O Devices:** Lists the **hardware devices currently utilized** by the process.
- **CPU Scheduling Information:** **Information about how the process is scheduled for execution** by the CPU.
- **Memory Management Information:** Details about **the memory allocated to the process**, such as base and limit registers, segment tables, and page information.
- **Process Control Block (PCB):** **A data structure that stores all the attributes of a process.** The PCB is essential for context switching and process management.

2.2.3 Process States

Each process in the operating system is tagged with a '**state**' variable—an **integer value that helps the operating system to decide what to do with the process.** It also indicates the nature of the current activity in a process. A process may be in one of the following states depending on the current activity of the process.

- **New:** A process is said to be in 'new' state if it is being created.
- **Ready:** A process is said to be in 'ready' state if it **is ready for the execution and waiting for the CPU** to be allocated to it.

- **Running:** A process is said to be in 'running' state if the CPU has been allocated to it and it is being executed.
- **Waiting:** A process is said to be in 'waiting' state (also called 'blocked' state) if it has been blocked by some event. Unless that event occurs, the process cannot continue its execution. Examples of such blocking events are completion of some I/O operation, reception of a signal, etc. Note that a process in waiting state is unable to run even if the CPU is available.
- **Terminated:** A process is said to be in 'terminated' state if it has completed its execution normally or it has been terminated abnormally by the operating system because of some error or killed by some other process.

Note: On a single processor system, only one process may be in running state at one time; however, in a multiprocessor system with m CPUs, at most m processes may be in running state at one time.

Each process undergoes change in state during its lifetime. The change in state of a process is known as **state transition** of a process. By and large, it is caused by the occurrence of some event in the system. There are many possible state transitions (see Figure 2.1) that may crop up. These transitions along with their possible causes are as follows:

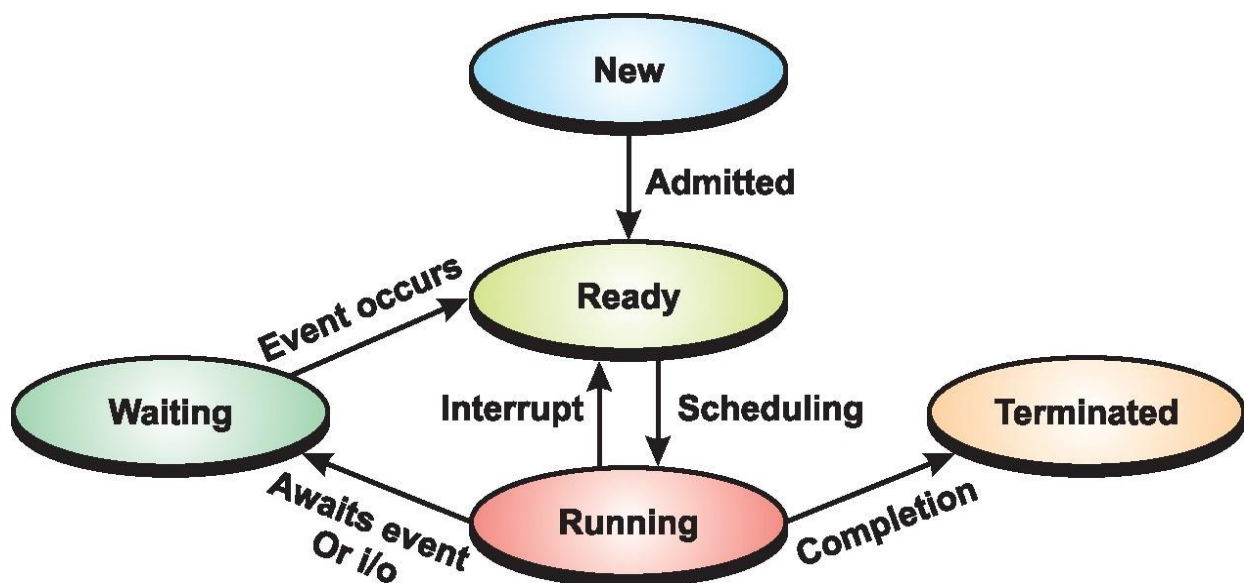


Fig. 2.1 Process State Transition Diagram

- **New → Ready:** This transition takes place if a **new process has been loaded into the main memory and it is waiting for the CPU to be allocated to it.**
- **Ready → Running:** This transition takes place if the **CPU has been allocated to a ready process and it has started its execution.**
- **Running → Ready:** This transition may occur if:
 - the **time slice** of the currently running process has **expired.**
 - In this case, the **CPU is preempted from the currently executing process** and allocated to some another ready process.
- **Running → Waiting:** This transition may take place if the currently running process
 - **needs to perform some I/O operation,**
 - **has to wait for a message or some action** from another process,
 - **requests for some other resource.**

In this case, the CPU gets freed by the process and can be allocated to some other ready process.

- **Running → Terminated:** This transition takes place if the currently running process
 - **has completed its task and** requests to the operating system for its termination,
 - **is terminated by its parent in case the function performed by it is no longer required,**
 - **is terminated by the kernel** because it has exceeded its resource usage limit or involved in a deadlock.

In this case, the CPU is preempted from the currently running process and allocated to some other ready process.

- **Waiting → Ready:** This transition takes place **if an event** (for example, I/O completion, signal reception, and synchronization operation) **for which the process was waiting, has occurred.**

2.2.4 Process Control Block (PCB)

To keep track of all the processes in the system, the operating system maintains a structurally organized table called **process table** that includes an entry for each process. This entry is called **process control block (PCB)**—a data structure created by the operating system for representing a process. A process control block stores descriptive information pertaining to a process such as its state, program counter, memory management information, information about its scheduling, allocated resources, accounting information, etc., that is required to control and manage a particular process. **The basic purpose of PCB is to indicate the progress of a process so far.** Some of the important fields stored in a PCB are as follows:

- **Process ID:** Each process is assigned a unique identification number called **process identifier (PID)** by the operating system at the time of its creation. PID is used to refer the process in the operating system.
- **Process state:** It stores the current state of a process that can be new, ready, running, waiting, or terminated.
- **Parent process ID:** It stores the `PID` of the parent, if the process has been created by some other process.
- **Child process IDs:** It stores the `PIDs` of all the child processes of a parent process.
- **Program counter:** **It contains the address of the instruction that is to be executed in the process next.** Whenever the CPU switches from one process to another, the program counter of the old process is saved so that the operating system could resume with the same instruction whenever the old process is restarted.
- **Event information:** **If the process is in waiting state** then this field contains the **information about the event for which the process is waiting** to happen. For example, if the process is waiting for an I/O device, then this field stores the `ID` of that device.
- **Memory management information:** It includes information

Process ID
State
Pointer
Priority
Program counter
CPU registers
I/O information
Accounting information
etc...

related to the memory configuration for a process such as **the value of base and limit registers, the page tables** (if paging memory management technique has been used) or the **segment tables** (if segmentation memory management technique has been used).

- **CPU registers:** They store the contents of **index registers, general purpose registers, condition code information**, etc., at the time when the CPU was last freed by the process or preempted from the process.
- **CPU scheduling information:** It includes the information used by **scheduling algorithms** such as the process **priority number** (in case the priority scheduling is to be used for the process), the **pointers to appropriate scheduling queues** depending upon the current state of the process, **the time when CPU was last allocated** to the process, etc.
- **I/O status:** It includes information like I/O devices allocated to a process, pointers to the files opened by the process for I/O, the current position in the files, etc.

2.3 PROCESS SCHEDULING

The **main objective of multiprogramming** is to **keep the jobs organized in such a manner that CPU has always one to execute**. This confirms that **CPU is utilized to the maximum** level by reducing its idle time. This purpose can be jolly well achieved by keeping the CPU busy at all the times. This implies that **some process must always be running on the CPU**.

However, when **two or more processes compete for the CPU at the same time** then a **choice has to be made** as to which process to allocate the CPU next. **This procedure** of determining the next process to be executed on the CPU is called **process scheduling and the module of the operating system that makes this decision** is called **scheduler**.

Scheduling Queues

For scheduling purposes, there exist **different queues** in the system; these are as follows:

- **Job queue:** As the processes enter the system for execution, they are massed into a queue called **job queue** (or **input queue**) on a mass storage device such as hard disk.
- **Ready queue:** From the job queue, the processes which are ready for the execution are shifted into the main memory. In the **main memory**, these processes are kept into a queue called **ready queue**. In other words, the ready queue contains all those processes that are waiting for the CPU.
- **Device queue:** For each I/O device in the system, a separate queue is maintained which is called **device queue**. The process that needs to perform I/O during its execution is kept into the queue of that specific I/O device; it waits there until it is served by the device.

Generally, both the **ready queue** and **device queue** are maintained as linked lists that contain PCBs of the processes in the queue as their nodes. Each PCB includes a pointer to the PCB of the next process in the queue (see Figure 2.2). In addition, the header node of the queue contains pointers to the PCBs of the first and the last process in the queue.

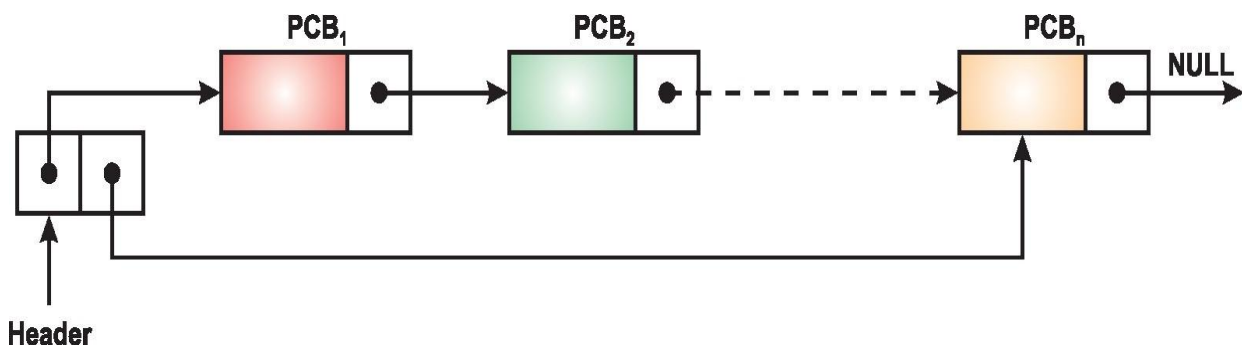


Fig. 2.2 Ready Queue and Device Queue Maintained as Linked List

Whenever a **process in the job queue** becomes ready to execute, it is **brought into the ready queue** where it waits for the CPU allocation. Once **CPU is allocated** to it (that is, the process switches to the **running** state), the following transitions may occur.

- If the process needs to perform some **I/O operation** during its execution, it is removed from the ready queue and **put into the**

appropriate device queue. After the process completes its I/O operation and is ready for the execution, it is switched from the device queue to ready queue.

- If an interrupt occurs, the CPU can be taken away from the currently executing process forcibly and the process has to wait until the interrupt is handled. After that the process is put back into the ready queue.
- If the time slice (in the case of time sharing systems) of the process has expired, the process is put back into the ready queue.
- If the process creates a new process and has to wait until the child process terminates, the parent process is suspended. After the execution of child process, it is again put back into the ready queue.
- If the process has successfully completed its task, it is terminated. The PCB and all the resources allocated to the process are deallocated.

All these transitions can be represented with the help of a queuing diagram as shown in Figure 2.3.

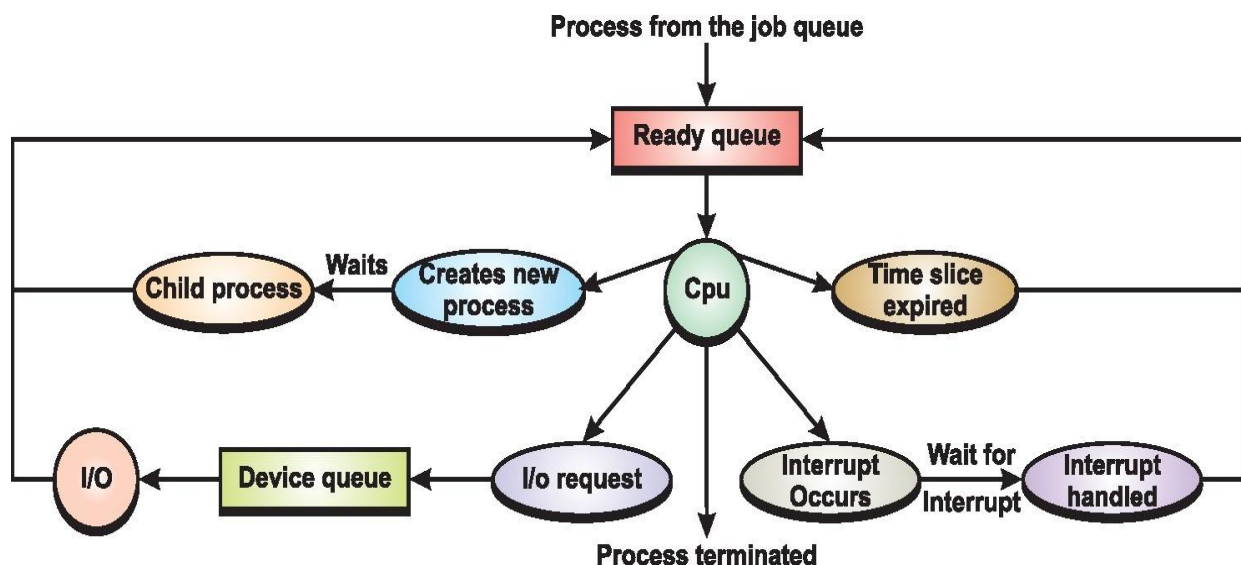


Fig. 2.3 Queuing Diagram

Note: In a single processor system, since there can be only one running process at a time, there is no need to maintain a queue for the running processes.

Types of Schedulers

The following types of schedulers (see [Figure 2.4](#)) may coexist in a complex operating system.

Long-term scheduler

- **Long-term scheduler**, also known as **job scheduler** or **admission scheduler**, works with the job queue.
- It selects the next process to be executed from the job queue and loads it into the main memory for execution.
- The long-term scheduler must select the processes in such a way that some of the processes are CPU-bound while others are I/O-bound. This is because if all the processes are CPU-bound, then the devices will remain unused most of the time. On the other hand, if all the processes are I/O-bound then the CPU will remain idle most of the time. Thus, to achieve the best performance, a balanced mix of CPU-bound and I/O-bound processes must be selected.
- The main objective of this scheduler is to control the degree of multiprogramming (that is, the number of processes in the ready queue) in order to keep the processor utilization at the desired level.
- For this, the long-term scheduler may admit new processes in the ready queue in the case of poor processor utilization or may reduce the rate of admission of processes in the ready queue in case the processor utilization is high.
- In addition, the long-term scheduler is generally invoked only when a process exits from the system.
- Thus, the frequency of invocation of long-term scheduler depends on the system and workload and is much lower than other two types of schedulers.

Short-term scheduler

- **Short-term scheduler**, also known as **CPU scheduler** or **process scheduler**, selects a process from the ready queue and allocates CPU to it.
- This scheduler is required to be **invoked frequently** as compared to the long-term scheduler.
- This is because generally a process executes for a short period and then it may have to wait either for I/O or for something else. At that time, CPU scheduler must select some other process and allocate CPU to it. Thus, the **CPU scheduler must be fast in order to provide the least time gap between executions.**

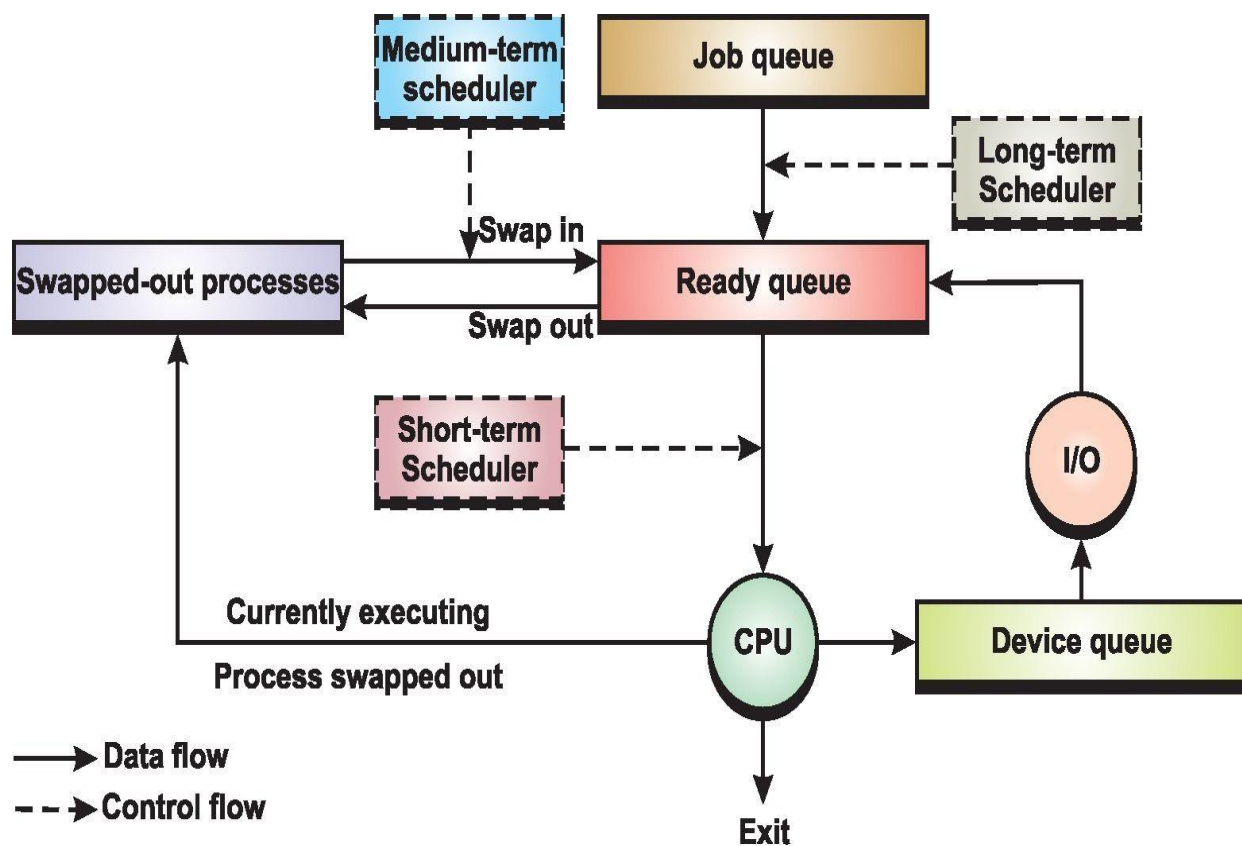


Fig. 2.4 Types of Schedulers

Medium-term scheduler

- **Medium-term scheduler**, also known as **swapper**, comes into play whenever a process is to be removed from the ready queue (or from the CPU in case it is being executed) thereby reducing the degree of multiprogramming.
- This process is stored at some space on the hard disk and later brought into the memory to restart execution from the point where it left off.
- This task of temporarily switching a process in and out of main memory is known as **swapping**.
- The medium-term scheduler selects a process among the partially executed or unexecuted swapped-out processes and swaps it in the main memory.
- The medium-term scheduler is usually invoked when there is some unoccupied space in the memory made by the termination of a process or if the supply of ready processes reduces below a specified limit.

Context Switch

Transferring the control of CPU from one process to another demands saving the context of the currently running process and loading the context of another ready process.

This mechanism of saving and restoring the context is known as **context switch**.

The portion of the process control block including the **process state, memory management information, and CPU scheduling information** together constitute the **context** (also called **state information**) of a process.

Context switch may **occur due to a number of reasons** some of which are as follows:

- The current **process terminates and exits** from the system.
- The **time slice** of the current process **expires**.
- The **process has to wait** for I/O or some other resource.
- Some **higher priority process enters** the system.
- The **process relinquishes the CPU** by invoking some system call.

Context switching is **performed in two steps**, which are as follows:

1. **Save context:** In this step, **the kernel saves the context** of the currently executing process **in its PCB of the process** so that it may **restore this context later** when its processing is done and the **execution of the suspended process can be resumed**.
2. **Load/Restore context:** In this step, **the kernel loads the saved context of a different process that is to be executed next**.

Note that **if the process to be executed is newly created and the CPU has not yet been allocated to it, there will be no saved context**. In this case, the kernel loads the context of the new process.

However, **if the process to be executed was in waiting state due to I/O or some other reason, there will be saved context that can be restored**.

One of the major detriments of using context switching is that **it incurs a huge cost to the system in terms of real time and CPU cycles because the system does not perform any productive work during switching**. Therefore, as far as possible, **context switching should be generally refrained from**, otherwise it can amount to reckless use of time. **Figure**

2.5 shows context switching between two processes P_1 and P_2 .

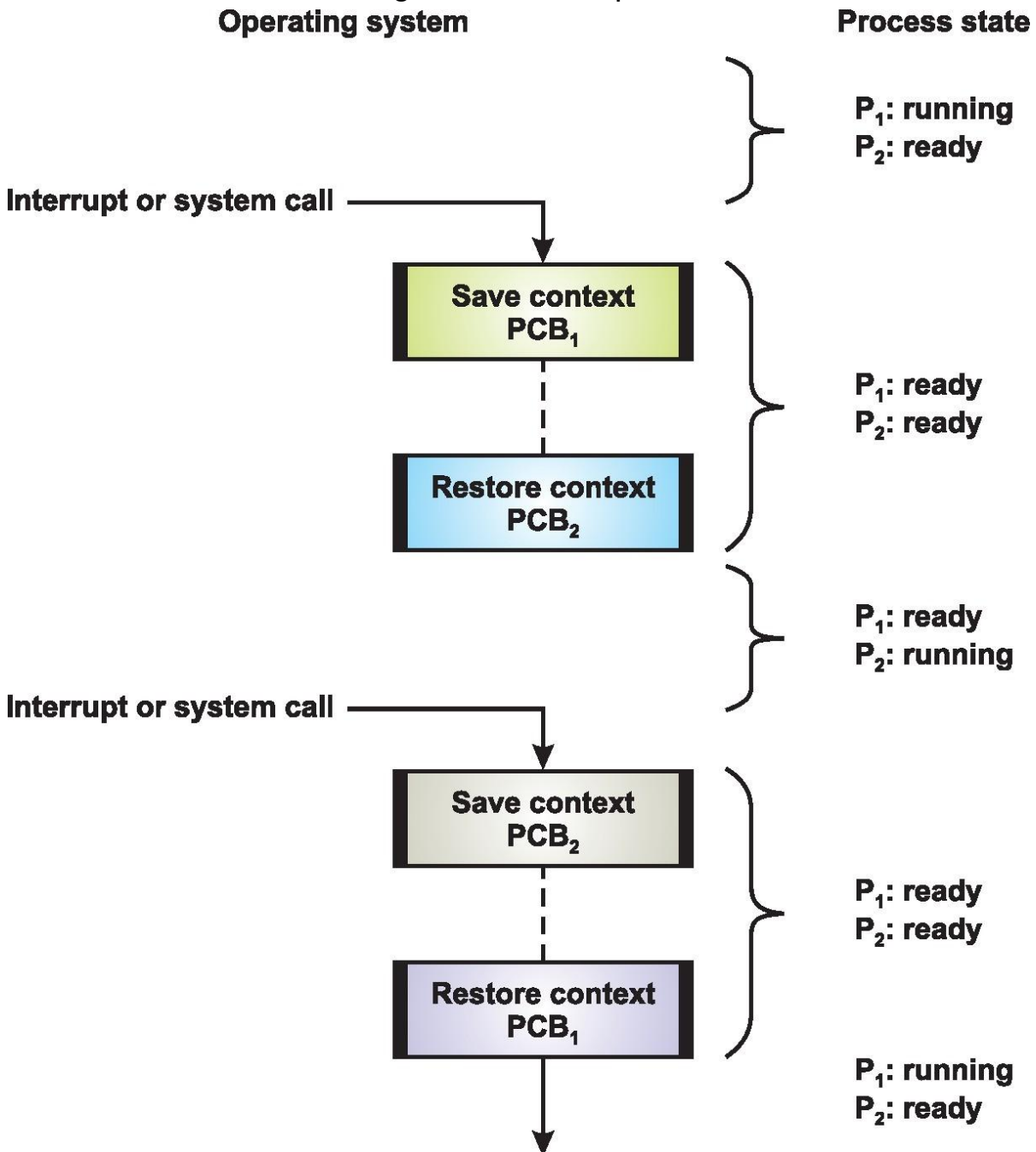


Fig. 2.5 Context Switching between Processes P_1 and P_2

2.4 COOPERATING PROCESSES

The processes that coexist in the memory at some time are called **concurrent processes**. The concurrent processes may either be

independent or cooperating.

The **independent** processes (also called **competitors**), as the name implies, do not share any kind of information or data with each other. They just compete with each other for the resources like CPU, and I/O devices that are required to accomplish their operations.

The **cooperating** (also called **interacting**) processes, on the other hand, need to exchange data or information with each other. In other words, we can say a cooperating process is the one that **can affect or be affected by the actions of other concurrent processes**.

The need for cooperation among processes arises because of the following reasons.

- Several processes may need access to same information. This requires the operating system to provide a means for concurrent access to the desired information.
- If a computer system has multiple processing elements (for example, multiple CPUs or multiple I/O channels), we can make a task to execute faster by breaking it into various subtasks and running each of them in parallel.
- The environment supporting cooperating processes will help a single user to carry out multiple tasks at the same time. For example, a single user may be opening, printing, and compiling at the same time.
- The system's functions can be divided into different processes and threads in order to construct the system in a modular fashion.

Independent Process	Cooperating Process
An independent process cannot affect or be affected by the execution of other processes .	A cooperating process can affect or be affected by the execution of other processes .
It executes independently without sharing data or resources with other processes.	It shares data, resources, or information with one or more processes.
No communication between processes is required.	Requires Inter-Process Communication (IPC) mechanisms such as shared memory

	or message passing.
Failure or termination of one independent process does not affect other processes.	Failure of one cooperating process may affect other cooperating processes.
Easier to implement since synchronization is not required.	More complex because synchronization and coordination are necessary.
Provides better isolation and security.	Enables resource sharing and improves system efficiency.
Example: Running a calculator application while listening to music. These applications work independently of each other.	Example: A word processor and a printer process. The word processor sends print jobs to the printer process, so they cooperate to complete the task.

Important Questions

What is a Process? Explain the various process attributes.

Explain the process state transition with the help of a neat diagram.

What is PCB? Explain its significance . Explain the attributes you find in a PCB with a neat diagram.

Explain the different scheduling queues (Job, ready, device).

Distinguish between different types of Scheduling queues.

Differentiate between different types of schedulers(Long, medium, short).

Differentiate between different types of queues for scheduling.

Differentiate between **Independent Processes** and **Cooperating Processes**.

What is context switching? What are the possible scenarios/reasons for a context switching?

What is Context switching. Explain the various steps involved with an example.